

58.1. La fonction `airFini()` — l'air est-il fini ?

`#include <airFini>` : `airFini(voix)` rend 1 quand l'air de cette voix est arrivé au bout.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `airFini(voix)` rend 1 quand l'air est arrivé au bout — ou que rien ne joue —, 0 tant qu'il joue. De quoi enchaîner : la fin de la musique de victoire lance l'écran suivant.

La ligne à écrire : `#include <airFini>`.

Son argument : la voix, 1 ou 2.

Ce qu'elle coûte : une vingtaine d'octets.

Essaie : relance l'air quand il est fini : il tourne en boucle, sans le 4^e argument.

On retrouve `airFini` dans 3 leçons

[59](#), [123](#), [136](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **906 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      74 o ← la console, à l'allumage - la mise en route de la console
main()         100 o ← démarrage - ta fonction
VBlank         11 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AvancerAirs    308 o ← VBlank - fait avancer la musique à chaque image
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   67 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         128 o ← CopierTuiles - 8 lettres sur 43
Notes          120 o ← AvancerAirs - la table des notes de musique
"ECOUTE"        6 o ← texte() - un texte gravé
"FINI"          4 o ← texte() - un texte gravé
COURT_0         8 o ← jouer() - une table gravée
```

Lettres gravées : CEFINOTU

Pas gravé (jamais appelé) : `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`

Fonctions du langage appelées :

```
jouer()   × 1   52 o sur place, dans main() + COURT_0 (8 o)
texte()   × 2   22 o sur place, dans main() + EcrireTexte (10 o)
airFini() × 1    5 o sur place, dans main()
image()   × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <airFini> : l'air est-il fini ? ----

#include <airFini>    // airFini() : 1 quand l'air est fini
#include <Air>
#include <jouer>
#include <texte>
```

2. UNE MÉLODIE — COURT

```
Air COURT = { "D05", "SOL4", "D04", "--" };
```

3. UNE VARIABLE — ECRIT

```
uint8_t ecrit = 0;
```

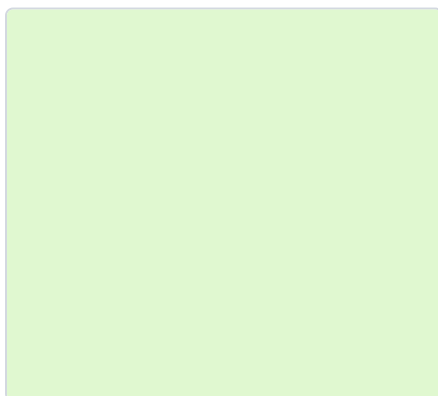
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(5, 4, "ECOUTE");
    jouer(1, COURT, 10);

    while (true) {
        image();
        if (airFini(1) && ecrit == 0) {    // l'air est arrivé au bout
            texte(7, 8, "FINI");
            ecrit = 1;
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60

4. Ce qu'on doit voir

ECOUTE ; trois notes qui descendent, puis FINI s'affiche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ pendant l'air, rien n'est écrit

✓ l'air fini, FINI s'affiche

Ensuite : **59. Une musique qui reprend au bout** — chapitre 8 : Le son. · gameboy3 — écrire une cartouche Game Boy en C++